



Recovery System

Database System Concepts, 6th Ed.

©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use



Failure Classification

- **Transaction failure :**
 - **Logical errors:** transaction cannot complete due to some internal error condition
 - **System errors:** the database system must terminate an active transaction due to an error condition (e.g., deadlock)
- **System crash:** a power failure or other hardware or software failure causes the system to crash.
 - **Fail-stop assumption:** non-volatile storage contents are assumed to not be corrupted by system crash
 - ▶ Database systems have numerous integrity checks to prevent corruption of disk data
- **Disk failure:** a head crash or similar disk failure destroys all or part of disk storage
 - Destruction is assumed to be detectable: disk drives use checksums to detect failures



Storage Structure

■ Volatile storage:

- does not survive system crashes
- examples: main memory, cache memory

■ Nonvolatile storage:

- survives system crashes
- examples: disk, tape, flash memory, non-volatile (battery backed up) RAM
- but may still fail, losing data

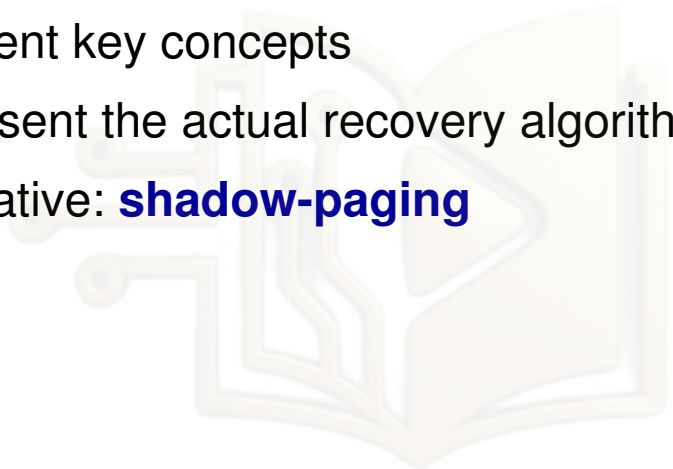
■ Stable storage:

- a mythical form of storage that survives all failures
- approximated by maintaining multiple copies on distinct nonvolatile media
- See book for more details on how to implement stable storage



Recovery and Atomicity

- To ensure atomicity despite failures, we first output information describing the modifications to stable storage without modifying the database itself.
- We study **log-based recovery mechanisms** in detail
 - We first present key concepts
 - And then present the actual recovery algorithm
- Less used alternative: **shadow-paging**





Log-Based Recovery

- A **log** is kept on stable storage.
 - The log is a sequence of **log records**, and maintains a record of update activities on the database.
- When transaction T_i starts, it registers itself by writing a $\langle T_i \text{ start} \rangle$ log record
- Before T_i executes **write**(X), a log record $\langle T_i, X, V_1, V_2 \rangle$ is written, where V_1 is the value of X before the write (the **old value**), and V_2 is the value to be written to X (the **new value**).
- When T_i finishes its last statement, the log record $\langle T_i \text{ commit} \rangle$ is written.
- Two approaches using logs
 - Deferred database modification
 - Immediate database modification



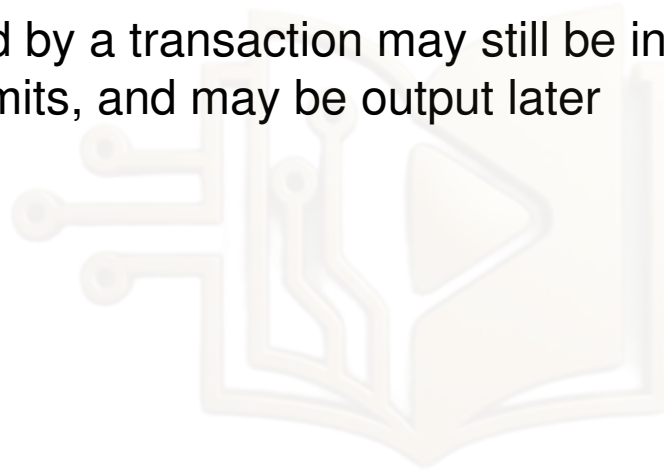
Immediate Database Modification

- The **immediate-modification** scheme allows updates of an uncommitted transaction to be made to the buffer, or the disk itself, before the transaction commits
- Update log record must be written *before* database item is written
 - We assume that the log record is output directly to stable storage
- Output of updated blocks to stable storage can take place at any time before or after transaction commit
- Order in which blocks are output can be different from the order in which they are written.
- The **deferred-modification** scheme performs updates to buffer/disk only at the time of transaction commit
 - Simplifies some aspects of recovery
 - But has overhead of storing local copy



Transaction Commit

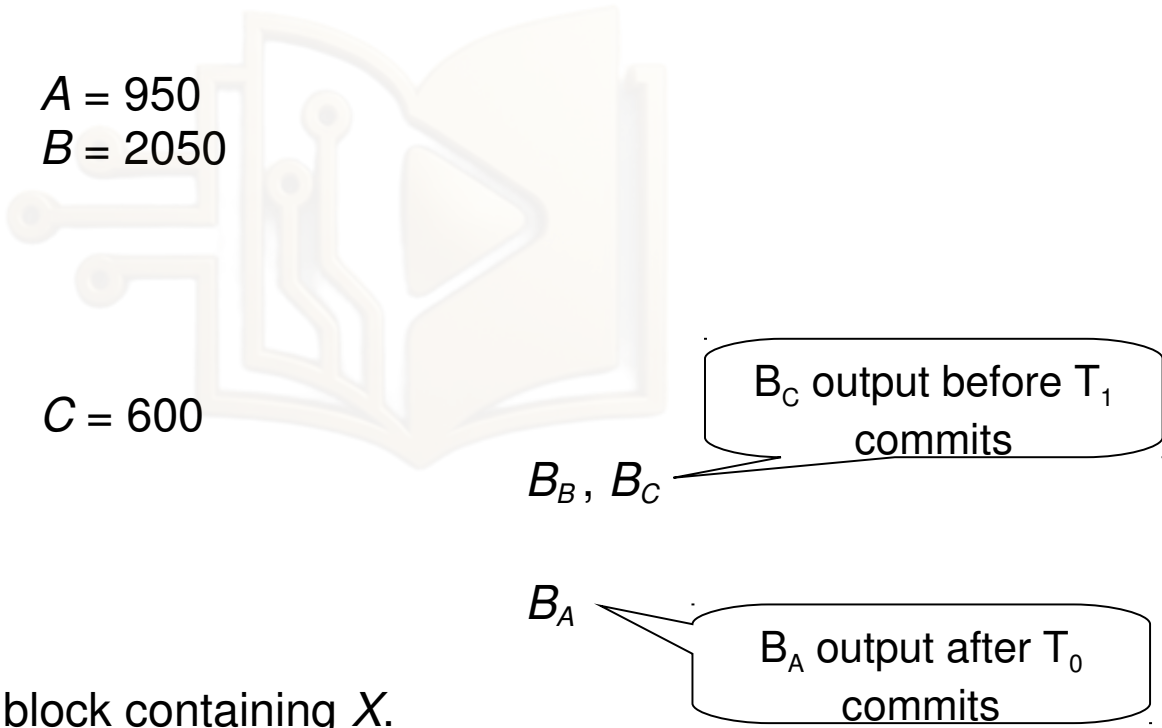
- A transaction is said to have committed when its commit log record is output to stable storage
 - all previous log records of the transaction must have been output already
- Writes performed by a transaction may still be in the buffer when the transaction commits, and may be output later





Immediate Database Modification Example

Log	Write	Output
$\langle T_0 \text{ start} \rangle$		
$\langle T_0, A, 1000, 950 \rangle$		
$\langle T_0, B, 2000, 2050 \rangle$		
	$A = 950$ $B = 2050$	
$\langle T_0 \text{ commit} \rangle$		
$\langle T_1 \text{ start} \rangle$		
$\langle T_1, C, 700, 600 \rangle$		
	$C = 600$	
$\langle T_1 \text{ commit} \rangle$		
■ Note: B_X denotes block containing X.		





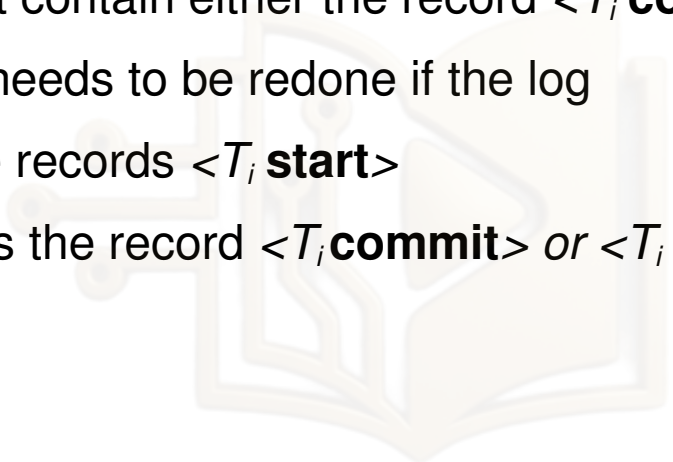
Undo and Redo Operations

- **Undo** of a log record $\langle T_i, X, V_1, V_2 \rangle$ writes the **old** value V_1 to X
- **Redo** of a log record $\langle T_i, X, V_1, V_2 \rangle$ writes the **new** value V_2 to X
- **Undo and Redo of Transactions**
 - **undo**(T_i) restores the value of all data items updated by T_i to their old values, going backwards from the last log record for T_i
 - ▶ each time a data item X is restored to its old value V a special log record $\langle T_i, X, V \rangle$ is written out
 - ▶ when undo of a transaction is complete, a log record $\langle T_i, \text{abort} \rangle$ is written out.
 - **redo**(T_i) sets the value of all data items updated by T_i to the new values, going forward from the first log record for T_i
 - ▶ No logging is done in this case



Undo and Redo on Recovering from Failure

- When recovering after failure:
 - Transaction T_i needs to be undone if the log
 - ▶ contains the record $\langle T_i \text{ start} \rangle$,
 - ▶ but does not contain either the record $\langle T_i \text{ commit} \rangle$ or $\langle T_i \text{ abort} \rangle$.
 - Transaction T_i needs to be redone if the log
 - ▶ contains the records $\langle T_i \text{ start} \rangle$
 - ▶ and contains the record $\langle T_i \text{ commit} \rangle$ or $\langle T_i \text{ abort} \rangle$





Immediate DB Modification Recovery Example

Below we show the log as it appears at three instances of time.

$\langle T_0 \text{ start} \rangle$
 $\langle T_0, A, 1000, 950 \rangle$
 $\langle T_0, B, 2000, 2050 \rangle$

(a)

$\langle T_0 \text{ start} \rangle$
 $\langle T_0, A, 1000, 950 \rangle$
 $\langle T_0, B, 2000, 2050 \rangle$
 $\langle T_0 \text{ commit} \rangle$
 $\langle T_1 \text{ start} \rangle$
 $\langle T_1, C, 700, 600 \rangle$

(b)

$\langle T_0 \text{ start} \rangle$
 $\langle T_0, A, 1000, 950 \rangle$
 $\langle T_0, B, 2000, 2050 \rangle$
 $\langle T_0 \text{ commit} \rangle$
 $\langle T_1 \text{ start} \rangle$
 $\langle T_1, C, 700, 600 \rangle$
 $\langle T_1 \text{ commit} \rangle$

(c)

Recovery actions in each case above are:

- (a) undo (T_0): B is restored to 2000 and A to 1000, and log records $\langle T_0, B, 2000 \rangle$, $\langle T_0, A, 1000 \rangle$, $\langle T_0, \mathbf{abort} \rangle$ are written out
- (b) redo (T_0) and undo (T_1): A and B are set to 950 and 2050 and C is restored to 700. Log records $\langle T_1, C, 700 \rangle$, $\langle T_1, \mathbf{abort} \rangle$ are written out.
- (c) redo (T_0) and redo (T_1): A and B are set to 950 and 2050 respectively. Then C is set to 600



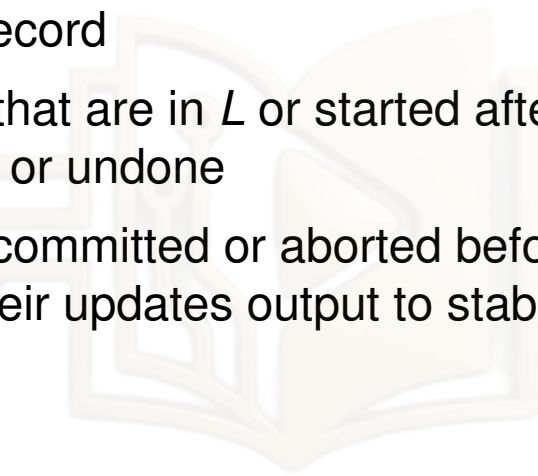
Checkpoints

- Redoing/undoing all transactions recorded in the log can be very slow
 1. processing the entire log is time-consuming if the system has run for a long time
 2. we might unnecessarily redo transactions which have already output their updates to the database.
- Streamline recovery procedure by periodically performing **checkpointing**
 1. Output all log records currently residing in main memory onto stable storage.
 2. Output all modified buffer blocks to the disk.
 3. Write a log record **< checkpoint L >** onto stable storage where L is a list of all transactions active at the time of checkpoint.
 - All updates are stopped while doing checkpointing



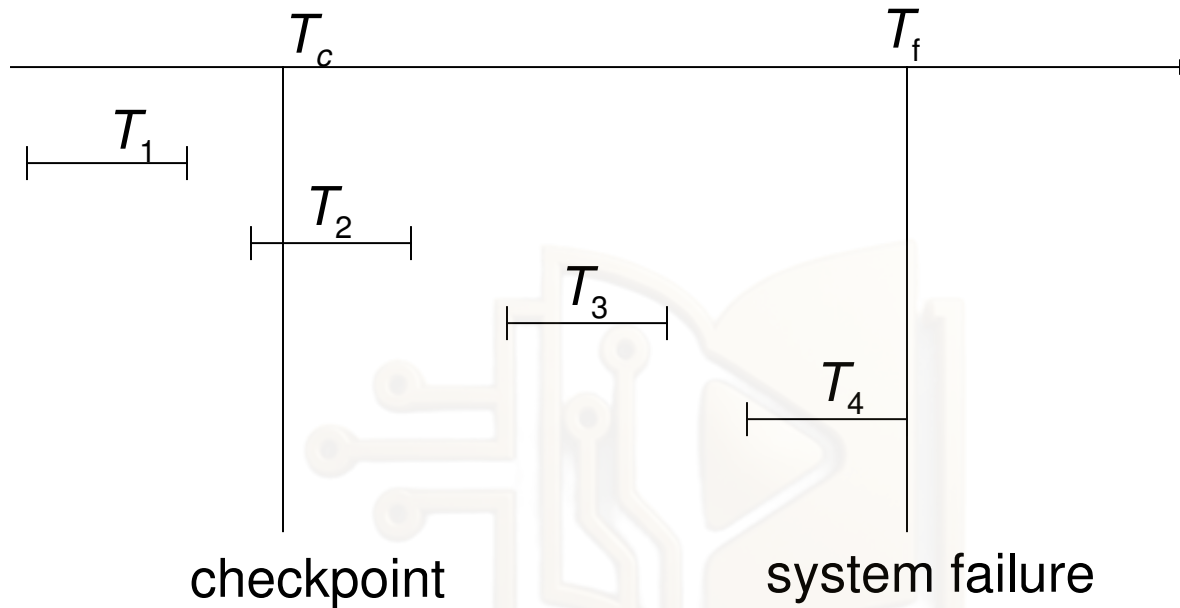
Checkpoints (Cont.)

- During recovery we need to consider only the most recent transaction T_i that started before the checkpoint, and transactions that started after T_i .
 1. Scan backwards from end of log to find the most recent **<checkpoint L >** record
 - Only transactions that are in L or started after the checkpoint need to be redone or undone
 - Transactions that committed or aborted before the checkpoint already have all their updates output to stable storage.





Example of Checkpoints



- T_1 can be ignored (updates already output to disk due to checkpoint)
- T_2 and T_3 redone.
- T_4 undone



Recovery Algorithm

- **So far:** we covered key concepts
- **Now:** we present the components of the basic recovery algorithm



Recovery Algorithm

- **Logging** (during normal operation):
 - $\langle T_i \text{ start} \rangle$ at transaction start
 - $\langle T_i, X_j, V_1, V_2 \rangle$ for each update, and
 - $\langle T_i \text{ commit} \rangle$ at transaction end
- **Transaction rollback (during normal operation)**
 - Let T_i be the transaction to be rolled back
 - Scan log backwards from the end, and for each log record of T_i of the form $\langle T_i, X_j, V_1, V_2 \rangle$
 - ▶ perform the undo by writing V_1 to X_j ,
 - ▶ write a log record $\langle T_i, X_j, V_1 \rangle$
 - such log records are called **compensation log records**
 - Once the record $\langle T_i \text{ start} \rangle$ is found stop the scan and write the log record $\langle T_i \text{ abort} \rangle$



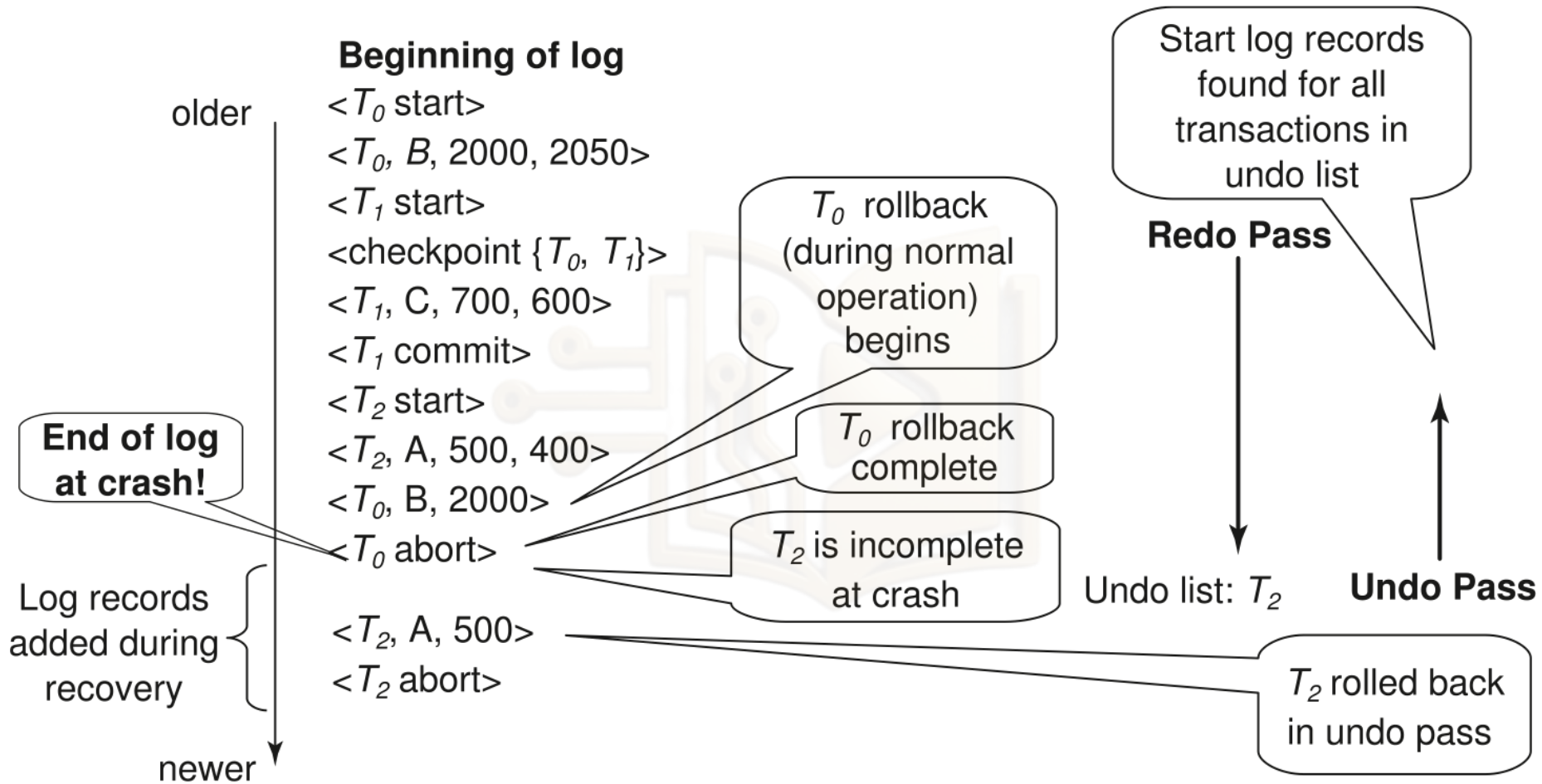
Recovery Algorithm (Cont.)

- **Recovery from failure:** Two phases
 - **Redo phase:** replay updates of **all** transactions, whether they committed, aborted, or are incomplete
 - **Undo phase:** undo all incomplete transactions





Example of Recovery





Failure with Loss of Nonvolatile Storage

- So far we assumed no loss of non-volatile storage
- Technique similar to checkpointing used to deal with loss of non-volatile storage
 - Periodically **dump** the entire content of the database to stable storage
 - No transaction may be active during the dump procedure; a procedure similar to checkpointing must take place
 - ▶ Output all log records currently residing in main memory onto stable storage.
 - ▶ Output all buffer blocks onto the disk.
 - ▶ Copy the contents of the database to stable storage.
 - ▶ Output a record **<dump>** to log on stable storage.



Recovering from Failure of Non-Volatile Storage

- To recover from disk failure
 - restore database from most recent dump.
 - Consult the log and redo all transactions that committed after the dump



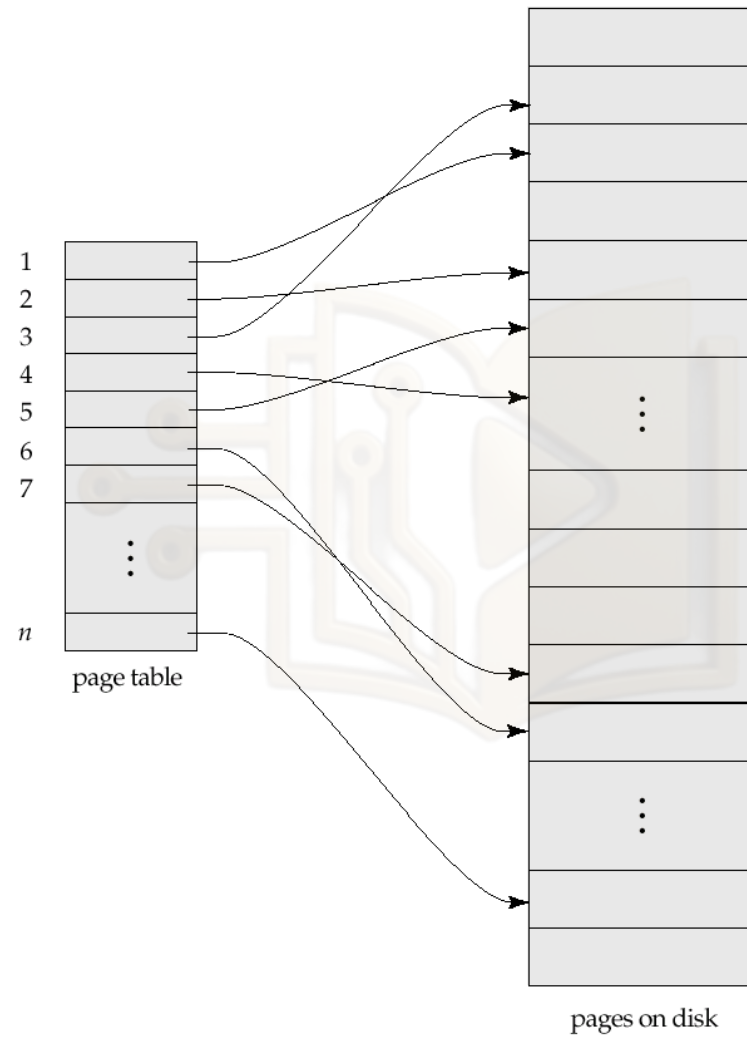


Shadow Paging

- **Shadow paging** is an alternative to log-based recovery; this scheme is useful if transactions execute serially
- Idea: maintain *two* page tables during the lifetime of a transaction –the **current page table**, and the **shadow page table**
- Store the shadow page table in nonvolatile storage, such that state of the database prior to transaction execution may be recovered.
 - Shadow page table is never modified during execution
- To start with, both the page tables are identical. Only current page table is used for data item accesses during execution of the transaction.
- Whenever any page is about to be written for the first time
 - A copy of this page is made onto an unused page.
 - The current page table is then made to point to the copy
 - The update is performed on the copy



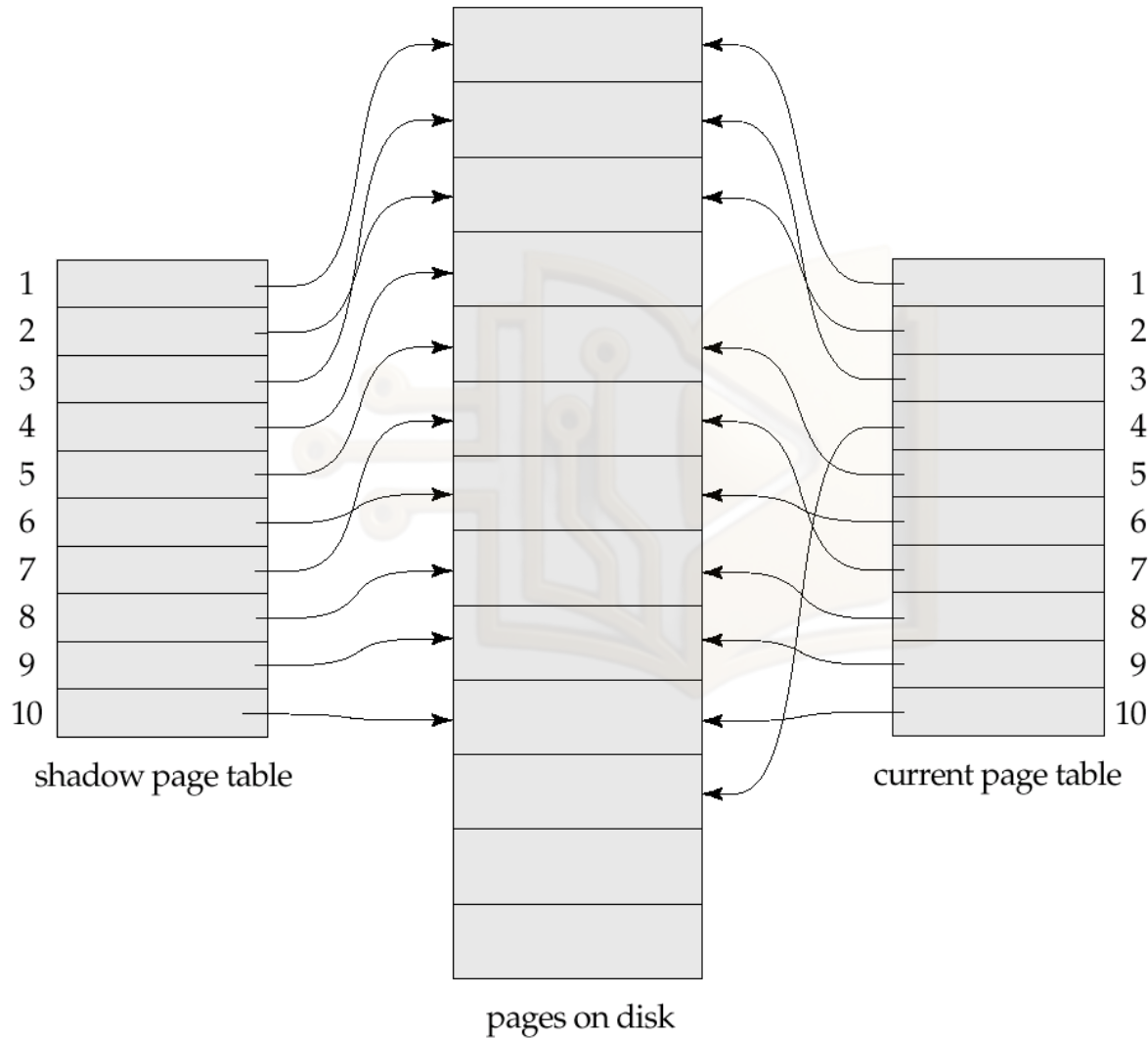
Sample Page Table





Example of Shadow Paging

Shadow and current page tables after write to page 4





Shadow Paging (Cont.)

- To commit a transaction :
 1. Flush all modified pages in main memory to disk
 2. Output current page table to disk
 3. Make the current page table the new shadow page table, as follows:
 - keep a pointer to the shadow page table at a fixed (known) location on disk.
 - to make the current page table the new shadow page table, simply update the pointer to point to current page table on disk
- Once pointer to shadow page table has been written, transaction is committed.
- No recovery is needed after a crash — new transactions can start right away, using the shadow page table.
- Pages not pointed to from current/shadow page table should be freed (garbage collected).



Show Paging (Cont.)

- Advantages of shadow-paging over log-based schemes
 - no overhead of writing log records
 - recovery is trivial
- Disadvantages :
 - Copying the entire page table is very expensive
 - ▶ Can be reduced by using a page table structured like a B+-tree
 - No need to copy entire tree, only need to copy paths in the tree that lead to updated leaf nodes
 - Commit overhead is high even with above extension
 - ▶ Need to flush every updated page, and page table
 - Data gets fragmented (related pages get separated on disk)
 - After every transaction completion, the database pages containing old versions of modified data need to be garbage collected
 - Hard to extend algorithm to allow transactions to run concurrently
 - ▶ Easier to extend log based schemes



Remote Backup Systems

Database System Concepts, 6th Ed.

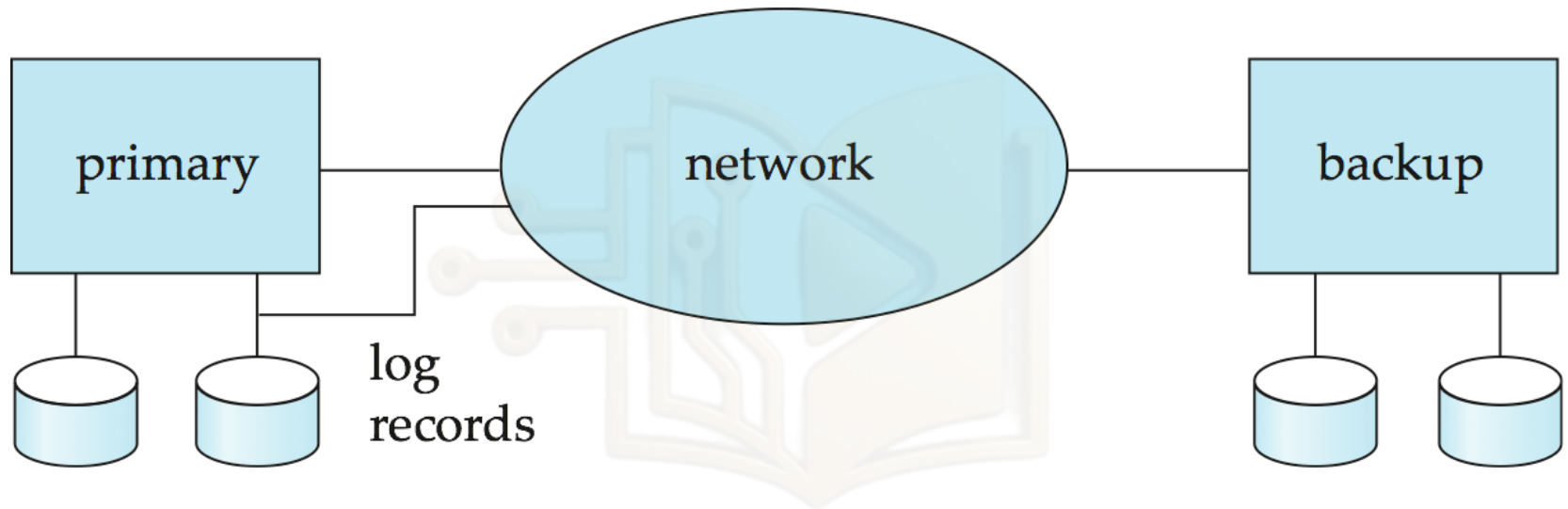
©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use



Remote Backup Systems

- Remote backup systems provide high availability by allowing transaction processing to continue even if the primary site is destroyed.





Remote Backup Systems (Cont.)

- **Detection of failure:** Backup site must detect when primary site has failed
 - to distinguish primary site failure from link failure maintain several communication links between the primary and the remote backup.
 - Heart-beat messages
- **Transfer of control:**
 - To take over control backup site first perform recovery using its copy of the database and all the log records it has received from the primary.
 - ▶ Thus, completed transactions are redone and incomplete transactions are rolled back.
 - When the backup site takes over processing it becomes the new primary
 - To transfer control back to old primary when it recovers, old primary must receive redo logs from the old backup and apply all updates locally.



Remote Backup Systems (Cont.)

- **Time to recover:** To reduce delay in takeover, backup site periodically processes the redo log records (in effect, performing recovery from previous database state), performs a checkpoint, and can then delete earlier parts of the log.
- **Hot-Spare** configuration permits very fast takeover:
 - Backup continually processes redo log record as they arrive, applying the updates locally.
 - When failure of the primary is detected the backup rolls back incomplete transactions, and is ready to process new transactions.
- Alternative to remote backup: distributed database with replicated data
 - Remote backup is faster and cheaper, but less tolerant to failure



Remote Backup Systems (Cont.)

- Ensure durability of updates by delaying transaction commit until update is logged at backup; avoid this delay by permitting lower degrees of durability.
- **One-safe:** commit as soon as transaction's commit log record is written at primary
 - Problem: updates may not arrive at backup before it takes over.
- **Two-very-safe:** commit when transaction's commit log record is written at primary and backup
 - Reduces availability since transactions cannot commit if either site fails.
- **Two-safe:** proceed as in two-very-safe if both primary and backup are active. If only the primary is active, the transaction commits as soon as its commit log record is written at the primary.
 - Better availability than two-very-safe; avoids problem of lost transactions in one-safe.